

TP Jour 4 — Sharding appliqué, Performances & Diagnostic (4 h)

Module MIA4 · Conception et intégration d'un SGBD NoSQL — IPSSI Montpellier Jour 4 / 5 — Shard key, targeted vs broadcast, Aggregation Pipeline, optimiseur, géospatial, profiler

Comment la journée est découpée

	Partie	Durée	Jeu de données
A	Sharding appliqué — vous choisissez une shard key et vous prouvez par la mesure qu'un mauvais choix ruine le cluster	≈ 1 h 15	<code>census.zips</code> (29 470 codes postaux, repris du Jour 3)
B	Performances & diagnostic — agrégation, qualité de données, optimiseur, géospatial, profiler	≈ 2 h 45	<code>citibike.trips</code> (10 000 trajets Citi Bike NYC)

La Partie A a été démontée hier en théorie (slides du Jour 3) : aujourd'hui on la **mesure**. Le cluster vous est livré **déjà assemblé** par un script — la plomberie n'est pas l'objet du TP, le choix de la clé l'est.

La sauvegarde/restauration et la sécurisation (RBAC, moindre privilège) font l'objet d'une démonstration du formateur en fin de séance, avec une fiche récapitulative fournie. Elles ne sont pas notées ici, mais **elles font partie du cahier des charges du projet de demain** (exigence n° 6) : suivez la démo.

Contexte professionnel — deux missions, dans cet ordre

Le matin, vous terminez la mission d'hier. Vous êtes toujours **ingénieur·e de production (SRE)** chez l'éditeur du **référentiel géodémographique américain** (`census.zips`, 29 470 codes postaux). Hier vous avez sécurisé la disponibilité avec un Replica Set. Reste la seconde échéance, celle que vous n'avez pas traitée : le produit annonce l'ouverture à **50 pays**, soit un volume ×50 qui ne tiendra plus sur une machine. La DSI attend de vous **un arbitrage écrit sur la shard key**, appuyé sur des mesures et non sur une opinion. (*Partie A*)

L'après-midi, vous changez de mission. Vous rejoignez la cellule *data* d'un opérateur de **vélos en libre-service**. La direction veut des chiffres sur l'usage — stations les plus sollicitées, profils d'usage, heures de pointe, durée moyenne des trajets — rafraîchissables tous les matins, et elle veut savoir si la base est prête à passer en ligne. On vous confie l'export brut du service **Citi Bike de New York** : **10 000 trajets réels** des 1^{er} et 2 janvier 2016 — durée, station de départ et d'arrivée avec leurs **coordonnées GeoJSON**, identifiant du vélo, type d'abonnement, année de naissance et genre de l'utilisateur. C'est le jeu officiel MongoDB `sample_training/trips`. (*Partie B*)

Les deux parties sont **indépendantes** : jeux de données différents, infrastructures différentes, livrables différents. Ne cherchez pas de lien entre les codes postaux américains et les vélos new-yorkais, il n'y en a pas. Ce qui les relie, c'est **vous** : deux façons de répondre « est-ce que ça tiendra ? », l'une par la topologie, l'autre par la mesure.

Attention — la donnée réelle est piégée. Ce jeu contient des durées de trajet aberrantes, un champ numérique stocké en chaîne **sur une partie exacte de la population**, et des noms de champs **contenant des espaces**. Aucune de ces anomalies n'est signalée : vous devez les **détecter par la requête**. Chaque réponse doit comporter la **commande exacte** et le **résultat exact**.

Objectifs pédagogiques (rappel du cours)

- Choisir une **shard key** et mesurer son impact : distribution, requêtes *targeted* vs *broadcast*, documents orphelins.
- Construire des **pipelines d'agrégation** : `$match`, `$group`, `$sort`, `$project`, `$unwind`, `$bucket`, `$lookup`, `$merge`.
- Comprendre pourquoi `$match` **en premier** et ce que l'**optimiseur** sait faire tout seul — et ce qu'il ne peut pas faire.
- Exploiter les **index géospatiaux** `2dsphere` : `$near`, `$geoWithin`, `$geoNear`.
- **Diagnostiquer** : `explain("executionStats")`, profiler et `system.profile`.

Pré-requis techniques

- Docker Desktop démarré, **4 Go de RAM** allouables (5 conteneurs : 4 pour le cluster shardé + 1 pour la Partie B).
- **Arrêtez d'abord les conteneurs du Jour 3** :
`docker compose -f ../../Jour3/apprenant/docker-compose.rs.yml down -v`
- Les fichiers fournis : `docker-compose.shard.yml`, `setup-shard.sh`, `docker-compose.yml`.
- Le fichier `zips.json` téléchargé hier (sinon, la commande de téléchargement est rappelée ci-dessous).

Livrables (Teams, avant 17h00)

1. `reponses_jour4.md` — commande + résultat exact pour chaque question, et R1→R4 rédigées.
 2. `bench_shard.md` — distributions avant/après, frontières de chunks, et vos 3 `explain()` *targeted* vs *broadcast*.
 3. `pipelines.js` — tous les pipelines des Parties B1 à B3, exécutables via `mongosh < pipelines.js`.
 4. `geo.js` — les requêtes géospatiales de la Partie B4.
 5. `diagnostic.md` — vos tableaux `explain()` avant/après index et vos extraits de `system.profile`.
-

PARTIE A — Sharding appliqué (≈ 1 h 15)

A0 — Monter le cluster (≈ 10 min)

Le script `setup-shard.sh` démarre les 4 conteneurs, initialise les 3 replica sets, enregistre les shards auprès du routeur et réduit la taille des chunks à 1 Mo. **Lisez-le avant de le lancer** : ses 5 étapes sont exactement ce que la Q1 vous demande d'expliquer. Comptez une **vingtaine de secondes**.

```
chmod +x setup-shard.sh && ./setup-shard.sh
docker exec -it mongos mongosh --eval 'sh.status()'
```

Sous Windows sans WSL (PowerShell ou CMD), le script shell ne s'exécute pas. Tapez les commandes à la main, dans cet ordre — ce sont les mêmes, et les lire vous prépare à la Q1 :

```
docker compose -f docker-compose.shard.yml up -d
(attendre ~15 s)
docker exec cfg1 mongosh --quiet --eval "rs.initiate({_id:'cfgRS',configsvr:true,members:[{_id:0,host:'cfg1'}]})"
docker exec shardA mongosh --quiet --eval "rs.initiate({_id:'shardA',members:[{_id:0,host:'shardA:27017'}]})"
docker exec shardB mongosh --quiet --eval "rs.initiate({_id:'shardB',members:[{_id:0,host:'shardB:27017'}]})"
(attendre ~15 s)
docker exec mongos mongosh --quiet --eval "sh.addShard('shardA/shardA:27017'); sh.addShard('shardB/shardB:27017');"
docker exec mongos mongosh --quiet config --eval "db.settings.updateOne({_id:'chunksize'},{$set:{value:1}},{u
```

- **Q1.** Décrivez le rôle de **chacun** des 4 conteneurs (`cfg1`, `shardA`, `shardB`, `mongos`). Lequel stocke la **carte** qui dit « tel intervalle de valeurs vit sur tel shard » ? Lequel n'héberge **aucune** donnée ? Enfin : le script réduit la taille des chunks de 128 Mo à 1 Mo — **pourquoi est-ce indispensable dans ce TP** et surtout, pourquoi serait-ce une **très mauvaise idée en production** ?

A1 — Sharder sur `state` (≈ 25 min)

```
# si vous n'avez plus zips.json :
curl -L -o zips.json \
  https://raw.githubusercontent.com/neelabalan/mongodb-sample-dataset/main/sample_training/zips.json

docker cp zips.json mongos:/tmp/zips.json
docker exec mongos mongoimport --db census --collection zips --drop --file /tmp/zips.json
```

```
// depuis mongos : docker exec -it mongos mongosh
sh.enableSharding("census")
use census
db.zips.createIndex({ state: 1 }) // obligatoire : la shard key doit être indexée
sh.shardCollection("census.zips", { state: 1 })
```

- **Q2.** Attendez ~30 s puis lancez `db.zips.getShardDistribution()`. Recopiez la sortie dans `bench_shard.md`. Combien de **chunks** ? Quel **pourcentage de documents** sur chaque shard ? La répartition est-elle équilibrée ?
- **Q3.** Inspectez les **frontières de chunks** :

```
const c = db.getSiblingDB("config");
const u = c.collections.findOne({ _id: "census.zips" }).uuid;
c.chunks.find({ uuid: u }).sort({ shard: 1 }).toArray().forEach(x => {
  const borne = v => (v && v.constructor && /^(MinKey|MaxKey)$/.test(v.constructor.name))
    ? v.constructor.name : v;
  print(x.shard + " [" + borne(x.min.state) + " -> " + borne(x.max.state) + "]");
})
```

(La fonction `borne` n'est là que pour l'affichage : concaténées naïvement dans une chaîne, les bornes spéciales s'afficheraient `[object Object]`. Regardez-les d'abord telles quelles avec `printjson` — c'est instructif.)

Que signifient les valeurs `MinKey` et `MaxKey` ? Sur quelle valeur d'État la coupure a-t-elle été faite ? Cette valeur est-elle le milieu de l'alphabet ? Qu'est-ce que le balancer a donc cherché à équilibrer ?

- **Q4. Découper plus, est-ce rééquilibrer ?** Le cluster ne s'est découpé qu'en 2. Forcez 4 coupures supplémentaires :

```
["FL", "MI", "NY", "TX"].forEach(s => sh.splitAt("census.zips", { state: s }))
```

Relancez `getShardDistribution()` après une minute.

- (a) Combien de chunks maintenant ?
- (b) Donnez le pourcentage de documents par shard avant (Q2) et après. De combien de points a-t-il bougé ?
- (c) Expliquez le résultat. (Comptez le nombre de codes postaux par État : `db.zips.aggregate([{$group:{$_id:"$state",n:{$sum:1}}},{ $sort:{$n:-1}},{ $limit:5}])`. Que peut faire le balancer quand un seul État pèse plus qu'un chunk entier ?)

A2 — Le piège du comptage (≈ 10 min)

- **Q5. La question d'écart.** Depuis mongos, exécutez les deux :

```
db.zips.countDocuments({})
db.zips.estimatedDocumentCount()
```

- (a) Donnez les deux nombres et **calculez l'écart**.
- (b) Comparez cet écart au nombre de documents affiché pour **l'un des shards** en Q2. Que remarquez-vous ?

- (c) Quel est le nom du phénomène ? (Indice : cherchez « orphaned documents » et le paramètre `orphanCleanupDelaySecs`.) Laquelle des deux commandes faut-il bannir sur un cluster shardé, et pourquoi l'autre est-elle plus coûteuse ?
- (d) Cherchez la valeur par défaut d' `orphanCleanupDelaySecs` et déduisez-en ce que donneraient ces deux commandes **15 minutes plus tard**. Formulez votre prédiction **maintenant**, par écrit. **Vérification en fin de Partie B** (le cluster doit être resté allumé — voir l'encadré en fin de Partie A) : relancez les deux commandes et comparez au chiffre que vous aviez prédit. En quoi une anomalie qui **disparaît d'elle-même** est-elle plus dangereuse en production qu'une anomalie permanente ?

A3 — Targeted vs broadcast : la démonstration qui compte (≈ 20 min)

- Q6. Analysez ces deux requêtes depuis mongos :

```
db.zips.find({ state: "NY" }).explain("executionStats") // filtre = la shard key
db.zips.find({ city: "NEW YORK" }).explain("executionStats") // filtre = un autre champ
```

Pour chacune, relevez dans `bench_shard.md` : le `stage` racine de `winningPlan`, la liste des shards interrogés, `nReturned` et `totalDocsExamined`.

- Q7. L'une est *targeted*, l'autre est *broadcast* (scatter-gather).
 - (a) Laquelle, et à quel signe précis le voyez-vous dans l' `explain` ? Citez le nom du `stage` et le contenu de `winningPlan.shards`.
 - (b) Calculez le rapport `totalDocsExamined / nReturned` pour la requête broadcast. Donnez-le.
 - (c) Extrapolez à un cluster de **20 shards et 500 millions de documents** : combien de machines cette requête mobiliserait-elle, et combien de documents seraient lus ? Formulez ce que cela dit de la scalabilité d'un cluster mal shardé.

A4 — La clé hachée, et le compromis (≈ 15 min)

```
sh.shardCollection("census.zips_hashed", { _id: "hashed" }) // collection encore vide
```

```
docker exec mongos mongoimport --db census --collection zips_hashed --file /tmp/zips.json
```

- Q8. `db.zips_hashed.getShardDistribution()` : combien de documents et quel pourcentage par shard ? Combien de chunks, **sans aucun `splitAt` manuel** ? Expliquez pourquoi le hachage donne d'emblée cette répartition (cherchez « *pre-splitting* » d'une clé hachée). Comparez aussi `countDocuments` et `estimatedDocumentCount` sur cette collection : l'écart de la Q5 existe-t-il ici ? Pourquoi ?
- Q9. **Le compromis, prouvé, puis arbitré**. Lancez sur la collection hachée **exactement la même requête métier** qu'en Q6 :

```
db.zips_hashed.find({ state: "NY" }).explain("executionStats")
```

- (a) Le `stage` racine est-il le même que pour `census.zip` ? **Formulez en une phrase le compromis fondamental du sharding** que cette comparaison démontre.
- (b) Remplissez ce tableau de décision. (Pour `zip`, réutilisez votre réponse à la Q4 du Jour 3 : ce champ est-il unique ? Pour la clé composée, demandez-vous ce qu'elle apporte à une requête qui ne filtre que sur `state`.)

Shard key candidate	Cardinalité	Distribution mesurée	Requêtes métier ciblées ?	Verdict
<code>{ state: 1 }</code>				
<code>{ _id: "hashed" }</code>				
<code>{ zip: 1 }</code>				
<code>{ state: 1, zip: 1 }</code>				

Laissez le cluster shardé ALLUMÉ pendant la Partie B.

Il ne gêne pas : ses 4 conteneurs sont publiés sur les ports **27119 à 27122**, l'instance de la Partie B utilise le **27017**. Aucun conflit.

Et surtout, il **doit** tourner : le nettoyage des documents orphelins de la Q5 est un compte à rebours **interne à MongoDB** (`orphanCleanupDelaySecs`). Un cluster arrêté ne compte pas — si vous l'éteignez, la vérification de la Q5(d) ne donnera jamais rien, même en revenant trois heures plus tard.

(Si votre machine peine réellement avec 5 conteneurs MongoDB, arrêtez plutôt `cfg1` en dernier recours — mais vous perdez la Q5(d), signalez-le dans votre rendu.)

PARTIE B — Performances & diagnostic (≈ 2 h 45)

B0 — Environnement et import (≈ 10 min)

```
docker compose up -d
docker compose ps

curl -L -o trips.json \
  https://raw.githubusercontent.com/neelabalan/mongodb-sample-dataset/main/sample_training/trips.json
wc -l trips.json          # attendu : 10000

docker cp trips.json mongo-j4:/tmp/trips.json
docker exec mongo-j4 mongoimport -u admin -p ipssi2025 --authenticationDatabase admin \
  --db citibike --collection trips --drop --file /tmp/trips.json
```

Connexion au shell (pensez à l'alias) :

```
docker exec -it mongo-j4 mongosh -u admin -p ipssi2025 --authenticationDatabase admin citibike
```

Point de contrôle B0 — `db.trips.countDocuments({})` doit renvoyer **10000**. Affichez un document avec `db.trips.findOne()` et **regardez attentivement les noms de champs** : `start station id`, `start station name`, `birth year`... Ils **contiennent des espaces**.

- **Q10.** Quelle conséquence concrète ces espaces ont-ils sur l'écriture de vos requêtes ? Donnez la syntaxe correcte pour (a) un filtre `find` sur `start station id`, (b) une référence à ce champ dans un `$group`. Que se passe-t-il si vous oubliez les guillemets ?
- **Q11.** Donnez la **plage temporelle** couverte par le jeu : `$min` de `start time` et `$max` de `stop time`. Le jeu s'annonce comme « janvier 2016 » — commentez ce que vous observez réellement.

B1 — Aggregation Pipeline : les fondamentaux (≈ 45 min) — fichier `pipelines.js`

- **Q12. Top 5 des stations de départ** par nombre de trajets (`$group` + `$sort` + `$limit`). Donnez les 5 noms et leurs comptes.
- **Q13. Répartition par type d'abonnement** (`usertype`) : nombre de trajets et **durée moyenne** (`tripduration`, en secondes) pour chacun. Le rapport entre les deux moyennes est frappant : **de combien**, et quelle hypothèse métier faites-vous ? (*Gardez ces deux moyennes : la Q21 vous demandera de les recalculer.*)
- **Q14. Trajets par jour** avec `$dateTrunc` sur `start time` (unité `day`), triés chronologiquement. Combien de jours obtenez-vous, et pourquoi ce résultat est-il cohérent avec votre réponse à la Q11 ?
- **Q15. Heure de pointe** : nombre de trajets par heure de départ (`$hour`). Donnez le **top 5**. Ce profil horaire ressemble-t-il à celui d'un usage domicile-travail ? Justifiez **avec la date du jeu** — quel jour de la semaine était le 1^{er} janvier 2016 ?
- **Q16. Distribution des durées** avec `$bucket` sur `tripduration`, bornes `[0, 300, 600, 1800, 3600, 1000000]`. Donnez les 5 effectifs. Quelle tranche est la plus peuplée ?
- **Q17. Boucles** : combien de trajets **repartent de la station où ils sont arrivés** ? (*Comparer deux champs du même document impose `$expr` dans un `$match`, ou un `$addFields` de comparaison.*)

B2 — Qualité de données et optimiseur (≈ 35 min)

Cette partie décide de la fiabilité de tous vos chiffres.

- **Q18. Le champ piégé.** Comptez les trajets dont `birth year` est de type **chaîne** (`$type: "string"`) et ceux de type **entier**. Puis **croisez** ce résultat avec `usertype` (un `$group` sur les deux). Que découvrez-vous ? (*Le nombre doit vous sauter aux yeux : comparez-le à votre réponse Q13.*) Expliquez pourquoi une requête `{ "birth year": { $lt: 1950 } }` est **silencieusement fausse**.

- **Q19.** Calculez l'**âge moyen** des usagers en 2016, en ne retenant que les années de naissance numériques. Donnez la moyenne, l'effectif retenu, et l'**âge du plus vieil usager**. Cette dernière valeur est-elle crédible ? Que feriez-vous en production de ce document ?
- **Q20. Les valeurs aberrantes.** Combien de trajets durent **plus de 3 heures** ? Plus de **24 heures** ? Affichez les 3 plus longs (durée + `usertype`). Proposez une explication métier.
- **Q21. La question d'écart.** Recalculez la durée moyenne par `usertype` (Q13) **en excluant les trajets de plus de 3 heures**.
 - (a) Donnez les nouvelles moyennes.
 - (b) **Calculez le pourcentage d'écart avec la Q13, pour chacun des deux `usertype`**. Les deux populations sont-elles affectées de la même façon ? Pourquoi ?
 - (c) Combien de trajets avez-vous exclus, et quel **pourcentage du jeu** cela représente-t-il ? Commentez le rapport entre ce pourcentage et l'écart calculé en (b).
 - (d) Laquelle des deux valeurs communiqueriez-vous à la direction, et pourquoi ?
- **Q22. `$match` en premier — vraiment ?** Écrivez ces deux pipelines qui renvoient la **même chose**, puis comparez-les avec `db.trips.explain("executionStats").aggregate([...])` :

```
// A
[ { $match: { usertype: "Subscriber" } },
  { $group: { _id: "$start station id", n: { $sum: 1 } } } ]
// B
[ { $group: { _id: { s: "$start station id", u: "$usertype" }, n: { $sum: 1 } } },
  { $match: { "_id.u": "Subscriber" } } ]
```

Relevez `totalDocsExamined` et le nombre de documents remontés du curseur pour chacun. **Les deux plans sont-ils différents ?** Cherchez « aggregation pipeline optimization » et expliquez ce que l'optimiseur a fait.

- **Q23. La limite de l'optimiseur.** Analysez maintenant :

```
[ { $group: { _id: "$start station id", n: { $sum: 1 } } },
  { $match: { n: { $gt: 50 } } } ]
```

Combien de documents traversent le `$group` cette fois ? **Pourquoi l'optimiseur ne peut-il rien faire ici ?** Combien de stations dépassent 50 départs ? Énoncez la règle générale qui en découle.

B3 — Matérialisation et jointure (≈ 20 min)

- **Q24. `$merge` — créer une collection dérivée.** Construisez une collection `stations` à partir de `trips` : une ligne par **station de départ**, avec `_id` = l'identifiant de station, son `nom`, sa `position` (le sous-document GeoJSON) et son nombre de `departes`.

```
db.trips.aggregate([
  { $group: { _id: "$start station id",
    nom: { $first: "$start station name" },
    position: { $first: "$start station location" },
    departs: { $sum: 1 } } },
  { $merge: { into: "stations", whenMatched: "replace" } }
])
```

Combien de stations obtenez-vous ? Donnez les 3 premières par nombre de départs.

- **Q25.** Quelle est la différence entre `$out` et `$merge` ? Laquelle des deux permet un **rafraîchissement quotidien incrémental** du tableau de bord de la direction, et pourquoi ?
- **Q26.** `$lookup`. Donnez le **top 5 des stations d'arrivée** (`end station id`) en affichant leur **nom** grâce à une jointure sur `stations`. Comparez ce classement à celui des départs (Q12) : une station apparaît-elle dans les deux ? Que peut signaler une station qui reçoit beaucoup plus de vélos qu'elle n'en émet ?

B4 — Index géospatial `2dsphere` (≈ 25 min) — fichier `geo.js`

Point de référence pour toute la partie : **Times Square**, coordonnées GeoJSON `[-73.9855, 40.7580]` (longitude **puis** latitude).

- **Q27. Sans index.** Avant de créer quoi que ce soit, lancez :

```
db.trips.find({ "start station location": { $near: {
  $geometry: { type: "Point", coordinates: [-73.9855, 40.7580] }, $maxDistance: 500 } } })
```

Recopiez l'erreur. Que dit-elle exactement, et pourquoi un index est-il **obligatoire** ici alors qu'il est seulement *conseillé* pour une requête classique ?

- **Q28.** Créez l'index `2dsphere` sur `start station location` puis relancez la requête. Combien de résultats ? Affichez les 5 premiers noms de station : dans quel ordre `$near` les renvoie-t-il ?
- **Q29. Le piège du comptage, encore.** Essayez maintenant de **compter** ces trajets :

```
db.trips.countDocuments({ "start station location": { $near: { ... } } })
```

Recopiez l'erreur et **expliquez-la** (indice : `countDocuments` est une *agrégation déguisée* ; que suggère le message ?). Trouvez l'opérateur de remplacement (`$geoWithin` + `$centerSphere` , dont le rayon s'exprime **en radians** : divisez les kilomètres par 6378,1) et donnez le nombre exact de trajets partant à **moins de 500 m** puis à **moins de 1 000 m** de Times Square.

- **Q30.** `$geoNear`. Écrivez un pipeline qui, sur la collection `stations` (Q24), renvoie les stations situées à moins de **1 km** de Times Square avec leur **nom**, leur **distance en mètres** (arrondie) et leur nombre de départs, triées par distance croissante. Vous devrez d'abord créer l'index adéquat sur `stations`. Combien de stations ? Quelle est la plus proche et à quelle distance ? (`$geoNear` doit être le **premier stage** du pipeline — pourquoi, à votre avis ?)

B5 — Diagnostic (≈ 30 min) — fichier `diagnostic.md`

- **Q31.** `explain()`. Sur la requête `db.trips.find({ "start station id": 476 })` :
 - (a) `explain("executionStats")` avant tout index : quel `stage` ? `totalDocsExamined` ? `nReturned` ?
 - (b) créez l'index adapté, relancez : quels sont les nouveaux `stage`, `totalKeysExamined`, `totalDocsExamined` ?
 - (c) Calculez le **ratio** `totalDocsExamined / nReturned` avant et après. Donnez les deux valeurs. Quelle valeur vise-t-on idéalement, et pourquoi ne l'atteint-on presque jamais sans projection ?
- **Q32. Le profiler.** Activez le profiler **sur toutes les opérations** de la base `citibike` :

```
db.setProfilingLevel(1, { slowms: 0 })
```

Exécutez une requête sur un champ **non indexé** (par exemple

`db.trips.find({ "end station name": "W 52 St & 9 Ave" })`) et une agrégation, puis **désactivez** le profiler (`db.setProfilingLevel(0)`). Interrogez `db.system.profile` : combien d'entrées ? Affichez pour chacune `op`, `ns`, `millis` et `planSummary`. Que vaut `planSummary` et qu'est-ce que cela vous apprend ?

- **Q33.** Quels sont les **trois niveaux** de profiling (0, 1, 2) ? Lequel utiliseriez-vous en production et avec quel `slowms` ? Citez **deux risques** à laisser le niveau 2 activé sur une base chargée. (*Regardez aussi la nature de la collection `system.profile` : `db.system.profile.stats().capped` — quelle conséquence ?*)
- **Q34.** Écrivez la requête qui, dans `system.profile`, **isole les opérations en COLLSCAN de plus de N millisecondes** — c'est exactement ce qu'on met dans un tableau de bord de production.

Partie C — Réflexion (≈ 20 min) — dans `reponses_jour4.md`

Comme hier : **une réponse qui ne cite aucun de vos chiffres ne vaut pas les points.** R2 et R3 exigent une manipulation supplémentaire.

- **R1. Le tableau de bord quotidien.** La direction veut son tableau de bord **rafraîchi chaque matin à 6 h**. Décrivez l'architecture que vous mettriez en place avec ce que vous avez appris aujourd'hui (`$merge`, index, profiler) plutôt que de recalculer les agrégations à chaque affichage. **Chiffrez le gain** : reprenez le `totalDocsExamined` de votre Q23 (agrégation complète sur `trips`) et comparez-le au nombre de documents de la collection `stations` (Q24). Quel est le rapport ? Quel est le **compromis accepté** ?
- **R2. La règle d'écriture des pipelines — vérifiée.** À partir des Q22 et Q23, formulez **votre** règle en trois phrases, en distinguant ce que l'optimiseur fait pour vous de ce qu'il ne peut pas faire. Puis **testez-la** : construisez un troisième pipeline où vous placez un `$match` après un `$project` qui supprime le champ filtré, et regardez l'`explain`. L'optimiseur remonte-t-il le `$match` cette fois ? Que vous apprend ce troisième cas sur la **frontière exacte** de ce qu'il sait faire ?

- **R3. Le chiffre unique, et son coût.** Reprenez les Q18 à Q21. Vous devez livrer un chiffre unique à la direction : « durée moyenne d'un trajet Citi Bike ».
 - (a) Rédigez la phrase que vous mettriez dans le rapport, avec **la valeur, l'effectif retenu et le critère d'exclusion explicite**.
 - (b) Calculez aussi la **médiane** (`$median`), disponible depuis MongoDB 7.0) sur le jeu **non filtré**. Comparez-la à vos deux moyennes (Q13 et Q21). Laquelle des trois valeurs est la plus robuste, et pourquoi ?
 - (c) En quoi une réponse sans précaution serait-elle **malhonnête** — et pas seulement imprécise ?
- **R4. `explain()` ou profiler ?** Comparez les deux : que voit l'un que l'autre ne voit pas ? Appuyez-vous sur ce que vous avez concrètement obtenu en Q31 et en Q32 (quel outil vous a donné quoi ?). Puis décrivez un incident de production — « l'appli est lente depuis 14 h » — et l'**ordre** dans lequel vous mobiliseriez : logs, `mongostat`, profiler, `explain`. Justifiez cet ordre par ce que chaque outil coûte et par ce qu'il permet d'éliminer comme hypothèse.

Pour aller plus loin (facultatif)

À traiter une fois les Parties A à C rendues. Chacun de ces points suppose **une mesure**, pas une intention.

- **B1. GridFS.** Un document BSON est plafonné à **16 Mo** ; GridFS découpe un fichier en *chunks* stockés dans deux collections. Envoyez `trips.json` dans GridFS (`mongofiles --db medias put /tmp/trips.json`), puis : relevez `length`, `chunkSize` et `uploadDate` dans `db.fs.files.findOne()` ; comptez `fs.chunks` et **retrouvez ce nombre par le calcul** ; donnez la taille du **dernier** chunk. Enfin, écrivez un script PyMongo (module `gridfs`) qui retélécharge le fichier et vérifie que la taille est identique. Question de fond : **pour un export de 7 Mo, GridFS est-il justifié ?** Quand l'utiliseriez-vous vraiment, et quand lui préféreriez-vous un stockage objet type S3 ?
- **B2. `$facet`.** Écrivez **un seul** pipeline qui renvoie d'un coup : le nombre total de trajets, la répartition par `usertype`, et la durée moyenne globale. Pourquoi `$facet` est-il plus efficace que trois requêtes séparées, et quelle est sa limite ? (*Regardez ce qui se passe pour l'usage des index dans les sous-pipelines.*)
- **B3. Index partiel et TTL.** Créez un index **partiel** ne portant que sur les trajets `Customer`, et un index **TTL** sur une collection `sessions`. Mesurez la taille des index avec `db.trips.stats().indexSizes` et comparez avec l'index complet équivalent. Donnez le gain en pourcentage.
- **B4. Collection time-series.** Recréez `trips` en **collection time-series** (`timeField: "start time"`, `metaField: "start station id"`), réimportez et comparez la taille de stockage avec `db.stats()`. Quel gain obtenez-vous, et **quelles opérations perdez-vous ?**
- **B5. Rejouer la démo du formateur.** Sauvegarde (`mongodump --db citibike --out /tmp/backup`), restauration dans une base de test (`mongorestore --nsFrom "citibike.*" --nsTo "citibike_test.*"`), puis création des deux utilisateurs `analyste` (rôle `read`) et `appli` (rôle `readWrite`) et test du moindre

privilège. **Le point que 90 % des équipes oublient** : après restauration, vérifiez la liste des index de `citibike_test.trips` — sont-ils revenus, et d'où viennent-ils ? C'est exactement l'exigence n° 6 du projet de demain.

Checklist finale avant rendu

`reponses_jour4.md` : Q1→Q34 avec commande + résultat exact, R1→R4 rédigées **en citant vos chiffres**

`bench_shard.md` : distributions `state` / hachée, frontières de chunks, les 3 `explain()`, le tableau de décision Q9(b)

`pipelines.js` et `geo.js` exécutables

`diagnostic.md` : tableau `explain()` avant/après, extraits de `system.profile`

Q5(d) vérifiée en fin de séance (les orphelins ont-ils disparu ?)

Conteneurs arrêtés proprement : `docker compose down` **et**

`docker compose -f docker-compose.shard.yml down -v`